



# **BabySteps**

CICLO: DAM

**Autor: Juan Luis Luque Romero**

**Tutor/a: Abelardo León González**

Repositorio oficial del proyecto: [BabySteps](#)

**Convocatoria: Septiembre/Diciembre 2025**



# 1. ÍNDICE

|                                                             |    |
|-------------------------------------------------------------|----|
| 1. Índice.....                                              | 2  |
| 2. Justificación.....                                       | 3  |
| 3. Introducción.....                                        | 4  |
| 4. Objetivos.....                                           | 5  |
| 5. Desarrollo.....                                          | 6  |
| A. Clase de Datos para la Guía (Consejo).....               | 7  |
| B. Clase de Datos para el Diario (Registro).....            | 8  |
| Figura 5: Navegación avanzada en la guía de desarrollo..... | 12 |
| Figura 6: Detalle de la ventana modal descriptiva.....      | 13 |
| 6. Conclusiones.....                                        | 14 |
| 7. Bibliografía.....                                        | 15 |
| 8. Anexos.....                                              | 16 |

## 2. JUSTIFICACIÓN

- **Por qué este proyecto:**

Este proyecto nace al observar que muchas aplicaciones dirigidas al cuidado de bebés resultan excesivamente complejas para los usuarios. En lugar de facilitar la experiencia, suelen incluir demasiadas funciones que no siempre son necesarias. Ante esta situación, surgió la idea de crear una aplicación más sencilla y directa, centrada únicamente en lo esencial y pensada para facilitar el día a día de madres y padres.

- **Valor personal:**

Como padre de un niño de 18 meses, conozco de primera mano algunos de los retos más comunes de esta etapa, como la falta de sueño o las dudas sobre la alimentación. Esta experiencia personal ha influido directamente en el enfoque y las decisiones tomadas en el desarrollo del proyecto.

- **Elección tecnológica:**

Se eligió Flutter como tecnología principal por permitir el desarrollo de una única base de código para Android e iOS, lo que reduce costes y tiempos de desarrollo. Además, ofrece un rendimiento cercano al nativo, garantizando una experiencia fluida para el usuario.

### 3. INTRODUCCIÓN

- Contexto del desarrollo móvil actual.

Hoy en día, es imposible ignorar que el móvil se ha convertido en el centro de casi todo lo que hacemos. Como desarrolladores, nos enfrentamos a un escenario donde el usuario ya no tiene paciencia: quiere apps que funcionen rápido, que sean bonitas y, sobre todo, que no fallen. Actualmente, el mercado está claramente dividido entre Android e iOS, y esto siempre ha sido un dolor de cabeza para los programadores. Hasta hace unos años, si querías sacar una app, tenías que programarla dos veces (una en Swift y otra en Kotlin), lo que aumentaba el trabajo y los errores.

En este escenario es donde entran frameworks como **Flutter**, que es el que he elegido para este proyecto. Lo que se busca ahora mismo en el sector no es solo "picar código", sino ser eficientes. Flutter permite que, con un solo equipo y un solo código, lleguemos a todo el mundo sin sacrificar ese "toque nativo" que tanto se agradece al deslizar una lista o pulsar un botón. Es una tecnología que Google está impulsando muchísimo y que está cambiando las reglas del juego en el desarrollo multiplataforma.

Además, hay que tener en cuenta el auge de las apps de "bienestar" o *wellness*. Ya no usamos el móvil solo para redes sociales; ahora lo usamos para gestionar cosas tan delicadas como la crianza de un hijo. En este contexto, la tendencia es ir hacia lo simple. El usuario ya está saturado de información, por lo que el desarrollo actual se centra en crear herramientas que quiten problemas en lugar de darlos.

**BabySteps** encaja aquí: aprovecha lo último en tecnología (Material Design 3 y programación reactiva) pero con un enfoque totalmente práctico y cercano, buscando resolver una necesidad real que tenemos los padres y las madres hoy en día, tener a un solo click la posibilidad de registrar notas y leer consejos rápidos.

- Presentación de BabySteps como una solución integral que une "conocimiento" (guía) y "acción" (diario).

La propuesta de **BabySteps** no nace simplemente como un ejercicio de programación, sino como una respuesta a lo que existe actualmente para los padres y madres. Tras analizar las opciones disponibles, me di cuenta de que la mayoría de las aplicaciones obligan al usuario a elegir: o son enciclopedias de lectura pesada o son simples cronómetros de registros fríos. BabySteps rompe con esto planteando una solución integral que combina dos pilares fundamentales: el "**conocimiento**" y la "**acción**".

El primer pilar, la **Guía de Consejos-Hitos**, funciona como el cerebro de la aplicación. Aquí no solo he volcado datos técnicos, sino que he estructurado la información de forma cronológica (de 0 a 18 meses) para que el padre o la madre encuentre exactamente lo que necesita según la edad de su hijo. Desde un punto de vista de desarrollo, esto se ha resuelto mediante una arquitectura de pestañas que permite una consulta rápida y visual. Es la parte informativa que aporta tranquilidad al usuario.

El segundo pilar es el **Diario de Registros**, que representa la parte ejecutiva de la app. Es aquí donde la aplicación se convierte en una herramienta activa. Mediante un sistema de botones de acceso rápido y formularios modales, el usuario puede anotar tomas, horas de sueño o cambios de pañal en apenas dos clics. Al unir estas dos funciones en un mismo flujo de trabajo, BabySteps evita que el usuario tenga que saltar de una app a otra. Mi objetivo con este diseño ha

sido crear un ecosistema donde la información (saber qué le toca hacer al bebé) y el registro (anotar qué ha hecho el bebé) convivan de manera natural, reduciendo significativamente la fatiga mental de los padres durante el primer año y medio de crianza.

## 4. OBJETIVOS

- **Generales:**

Crear una app funcional y escalable.

- **Específicos:**

- o Diseñar una interfaz intuitiva con Material Design 3.
- o Implementar un sistema de registro de eventos en tiempo real.
- o Categorizar hitos-consejos del desarrollo de 0 a 18 meses.

## 5. DESARROLLO

El desarrollo técnico de **BabySteps** se ha basado en el *framework* **Flutter**, utilizando el lenguaje de programación **Dart**. La arquitectura se fundamenta en el concepto de "árbol de widgets", donde cada elemento de la interfaz es un objeto que responde a un estado específico.

Para la gestión de datos, se han implementado modelos de clase orientados a objetos. La clase Registro permite encapsular la información crítica de cada actividad del bebé, incluyendo el marcado de tiempo (*timestamp*) capturado mediante la librería DateTime de Dart. Para la visualización de los consejos, se ha diseñado un componente dinámico que muestra la información de forma estructurada según la etapa de desarrollo.

Un aspecto crítico fue la implementación del StatefulWidget dentro de los menús contextuales. Esta técnica permite que el usuario interactúe con el formulario de registro (cambiando el tipo de evento o la nota) y vea los cambios reflejados de forma inmediata sin necesidad de reconstruir toda la pantalla principal, optimizando así el consumo de recursos del dispositivo.

### ● 5.1. Requisitos del sistema:

Para poner en marcha este proyecto, se optó por una decisión técnica estratégica: utilizar un entorno de desarrollo basado en la nube. Principalmente se ha trabajado con **Zapp.run**, una herramienta específica para Flutter que funciona sobre el navegador. Esto evitó conflictos iniciales con variables de entorno y configuraciones pesadas de SDK locales, permitiendo un flujo de trabajo ágil desde el primer día. Como apoyo para la revisión de la estructura de archivos y depuración, se utilizó **Visual Studio Code**.

A nivel de *hardware*, no ha sido necesario un equipo de altas prestaciones, ya que Flutter es eficiente en su compilación web. Lo fundamental ha sido el **SDK de Flutter** y el motor de **Dart**. Durante la configuración, la tarea más relevante fue la gestión del archivo pubspec.yaml, donde se incluyó la dependencia intl para la gestión profesional de formatos de fecha y hora, garantizando que el diario de cuidados sea preciso y legible.

El control de versiones y el repositorio central del código fuente se gestionan a través de GitHub, facilitando el acceso al histórico de desarrollo en el siguiente enlace: <https://github.com/JLLR85/TSDAM-BabySteps.git>.

### ● 5.2. Arquitectura de la aplicación:

La arquitectura de **BabySteps** se centra en la "Gestión de Estado". La aplicación no es estática; debe persistir la información durante la sesión de uso. Para solucionar esto de manera eficiente, se ha programado una lógica de **Listas Dinámicas**. Se ha definido un objeto List<Registro> registros que actúa como el núcleo de datos de la pestaña de Diario.

Otro punto clave es el **Filtrado de Contenido**. Para evitar saturar al usuario con una lista infinita de texto, se programó un sistema donde la interfaz renderiza únicamente los hitos-consejos relevantes. La comunicación entre widgets (o *widget communication*) fue el mayor reto técnico, logrando que el estado de la aplicación se actualice de forma reactiva: al guardar un nuevo evento en el formulario flotante, la lista del diario se actualiza instantáneamente gracias al método setState().

- **5.3. Modelado de datos:**

Se ha utilizado la **Programación Orientada a Objetos (POO)** para asegurar que los datos mantengan una estructura íntegra y evitar errores de ejecución en tiempo de ejecución (*runtime errors*).

**A. Clase de Datos para la Guía (Consejo):** Encargada de gestionar el contenido informativo de lectura, organizando los hitos por etapas mensuales.

Dart

```
class Consejo {  
  final String etapa;      // Ejemplo: "0-1 meses"  
  final String titulo;     // Ejemplo: "Reflejo de succión"  
  final String descripcion; // El contenido del consejo  
  final String categoria;  // Ejemplo: "Alimentación" o "Sueño"  
  
  Consejo({  
    required this.etapa,  
    required this.titulo,  
    required this.descripcion,  
    required this.categoria,  
  });  
}
```

**Figura 1:** Definición de la clase Consejo para la estructura de la guía.

**B. Clase de Datos para el Diario (Registro):** Es una clase dinámica que incluye campos de tipo String para la categoría y observaciones, y un marcador temporal para la precisión cronológica.

Dart

```
class Registro {  
  final String tipo;          // Ejemplo: "Toma", "Sueño", "Pañal"  
  final DateTime hora;        // Fecha y hora del evento  
  final String observacion;   // Notas adicionales del usuario  
  
  Registro({  
    required this.tipo,  
    required this.hora,  
    required this.observacion,  
  });  
}
```

**Figura 2:** Modelo de datos para la gestión de entradas en el diario.

La implementación de estos modelos permite que, más adelante en el código, podamos realizar operaciones como el filtrado por fecha o la categorización por tipos de icono, garantizando una base de código limpia y escalable para futuras versiones de la aplicación.

#### ● 5.4. Diseño de Interfaz (UI/UX):

A la hora de diseñar la interfaz de **BabySteps**, mi prioridad absoluta ha sido la **usabilidad bajo condiciones de estrés**. Como padre, sé que muchas veces tienes que usar el móvil con una sola mano mientras sostienes al bebé con la otra. Por eso, me he alejado de menús complicados y me he centrado en un diseño basado en **Material Design 3**, buscando que los elementos más importantes (como el botón de registro) estén siempre al alcance.



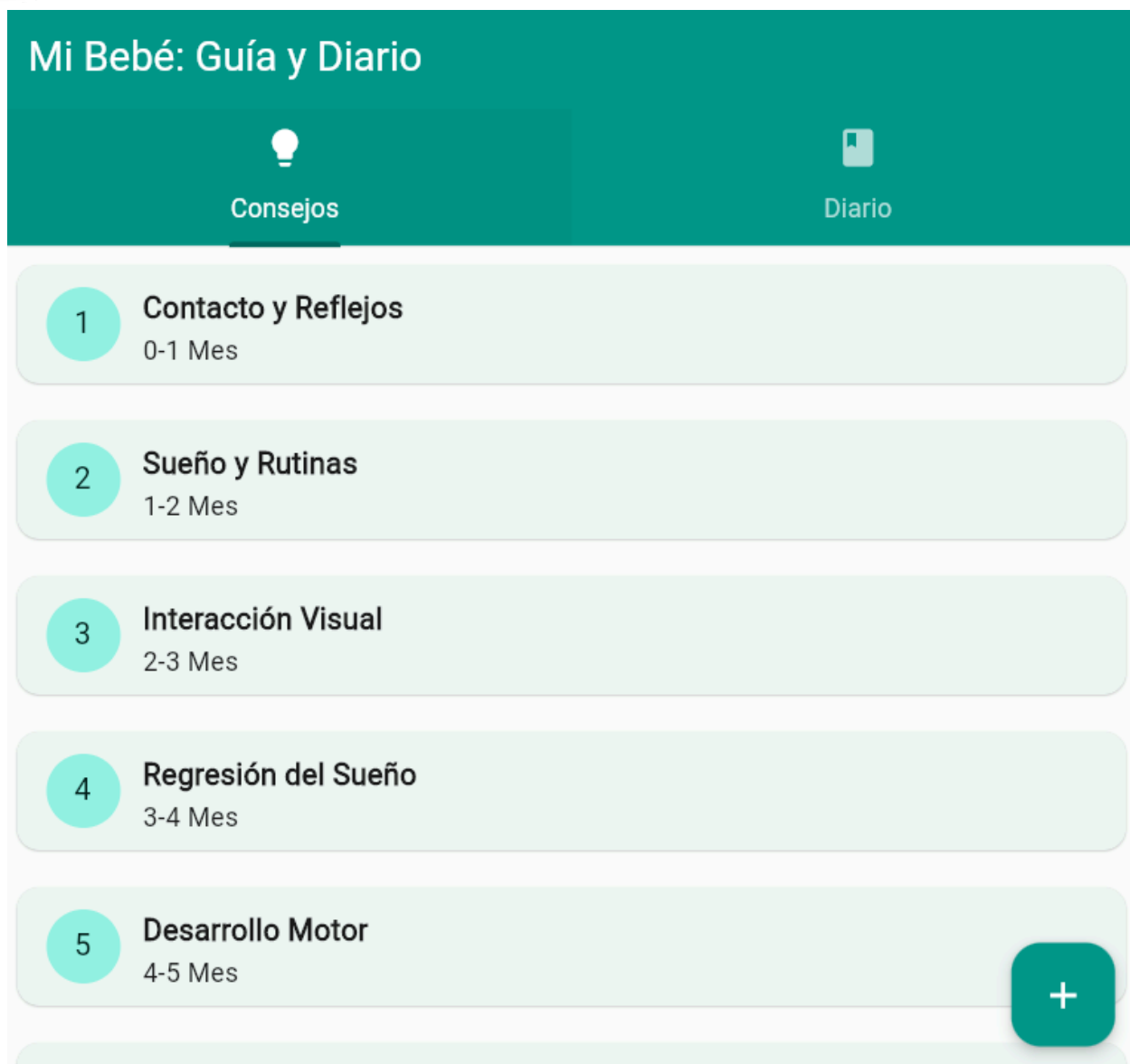


Figura 3: Vista principal de la pestaña "Consejos" de BabySteps, donde se implementa un ListView con tarjetas interactivas para el seguimiento de hitos del desarrollo (0-5 meses).

Como se puede apreciar en la imagen anterior, la interfaz utiliza un componente **TabBar** en la parte superior para separar de forma clara las dos funcionalidades núcleo del proyecto. Cada elemento de la lista está envuelto en un widget **Card** con elevación mínima, siguiendo las pautas de diseño moderno. Se ha priorizado la legibilidad mediante el uso de avatares numéricos con colores suaves, permitiendo que el usuario identifique rápidamente la progresión mensual del bebé sin saturar la vista con elementos innecesarios.

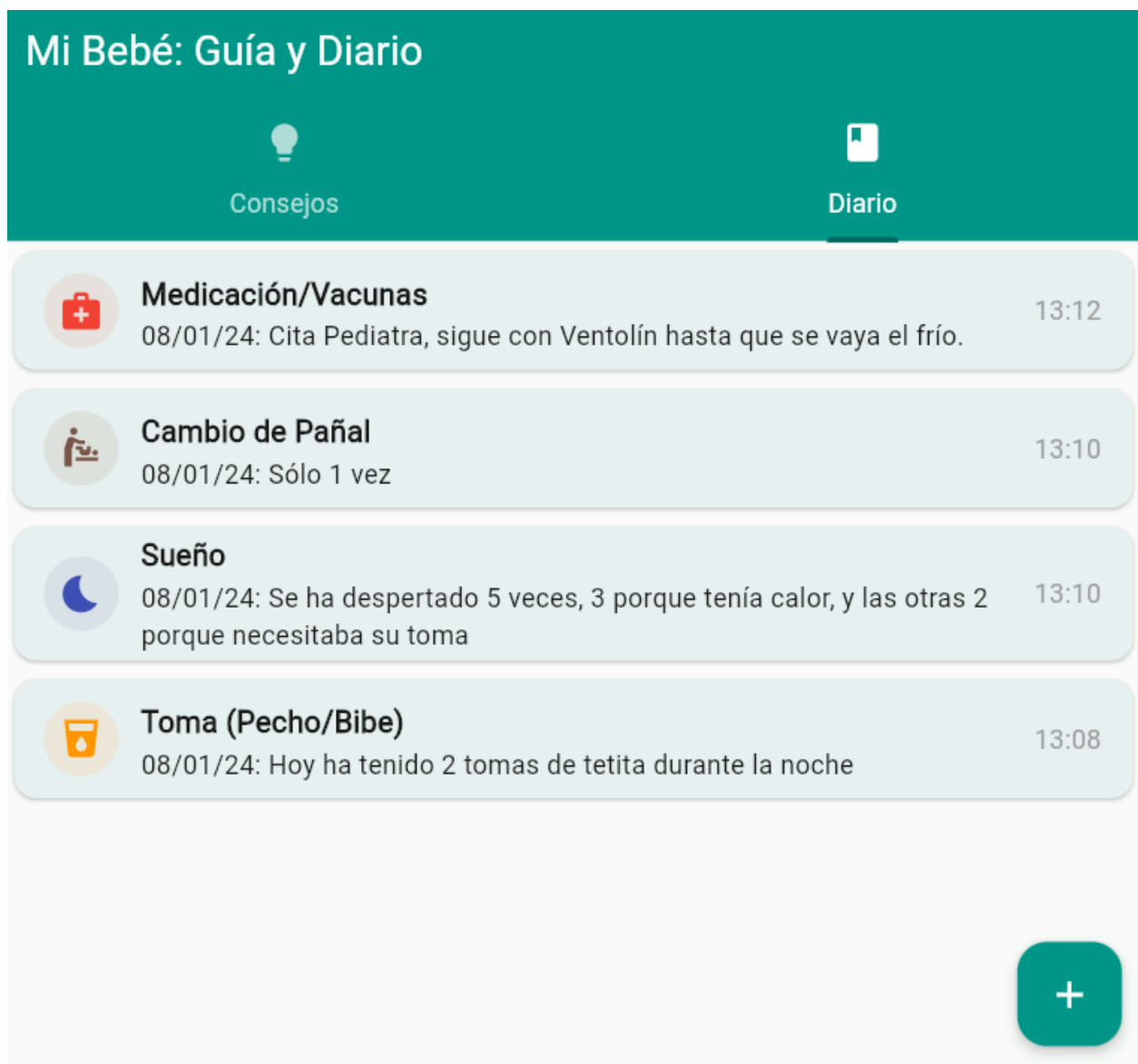


Figura 4: Vista de la sección "Diario" donde se muestran los registros de cuidados ordenados cronológicamente con iconos personalizados por categoría.

En la Figura 4 se observa la implementación práctica del diario de cuidados. Cada registro se genera de forma dinámica y se presenta en una tarjeta que incluye la hora exacta de la actividad y una breve descripción. El uso de iconos cromáticos (rojo para medicación, azul para sueño, naranja para alimentación) permite que, ante una situación de cansancio extremo, el usuario localice la última toma de un simple vistazo. Técnicamente, esta vista utiliza un **ListView.builder**, lo que optimiza el consumo de memoria RAM al renderizar únicamente los elementos visibles en pantalla.

He optado por una paleta de colores basada en el teal (verde azulado) y tonos suaves, aunque el diseño es compatible con esquemas tipo deepPurple. Estas tonalidades transmiten tranquilidad y cuidado, esencial en una aplicación infantil. Para la sección de la Guía de Hitos, se han implementado "Cards" con bordes redondeados e iconografía clara, facilitando la identificación del contenido antes de leer el título. Además, se ha añadido un degradado visual en la cabecera para aportar un acabado moderno y profesional.

En cuanto a la **Experiencia de Usuario (UX)**, el mayor reto fue la gestión de formularios. Para evitar que el usuario pierda el contexto cambiando de pantalla, se ha utilizado un `showModalBottomSheet`. Esta ventana emerge desde la parte inferior, permitiendo rellenar datos rápidamente. Para mejorar la accesibilidad, se han ajustado los tamaños de fuente a un mínimo de 14 puntos y se han definido márgenes generosos, evitando "toques accidentales" en dispositivos móviles.

## ● 5.5. Lógica de Funciones y Gestión de Eventos

En este apartado se detalla el funcionamiento lógico interno, el cual garantiza la interactividad de **BabySteps**. El núcleo de la gestión de eventos reside en el `FloatingActionButton`. Al activarse, este no redirige a una nueva ruta, sino que despliega un componente `showModalBottomSheet`, manteniendo la persistencia visual de la pantalla principal.

Dentro de este flujo, la aplicación gestiona tres procesos críticos de datos:

**-Captura de Categoría:** Mediante un componente `DropDownButton`, el sistema asigna un valor a la variable de estado, categorizando el evento (Toma, Sueño, etc.).

**-Validación y Control de Texto:** Se implementó un `TextEditingController` para capturar las observaciones del usuario. La lógica vincula estas cadenas al objeto `Registro` solo tras validar la entrada.

**-Marcado Temporal (Timestamping):** Se utiliza la clase `DateTime.now()` junto con la librería `intl` para registrar el instante exacto del evento, asegurando la precisión cronológica del diario.

Tras la confirmación del usuario, se invoca la función `setState()`. Este método es fundamental en el desarrollo con Flutter, ya que notifica al *framework* que el estado interno ha cambiado, forzando un redibujado selectivo de los widgets y mostrando el nuevo registro de forma instantánea sin necesidad de recargar la aplicación.

## ● 5.6. Pruebas de Sistema y Validación

Para garantizar que la aplicación cumple con los estándares de fiabilidad, se han ejecutado diversos protocolos de prueba:

**-Pruebas de Carga de Contenido:** Se ha verificado que la transición entre las pestañas de "Consejos" y "Diario" sea fluida. Se comprobó que el filtrado dinámico de hitos responde correctamente a los índices de edad seleccionados.

**-Pruebas de Persistencia Temporal:** Se validó que el formato de 24 horas y la ordenación cronológica inversa (los registros más nuevos aparecen primero) funcionen correctamente en distintos horarios.

**-Pruebas de Interfaz (Responsive Design):** Se ha testeado la aplicación en diferentes resoluciones. El uso de widgets de diseño flexible como `Expanded` y `ListView.builder` evita errores de desbordamiento de píxeles (*pixel overflow*) en dispositivos de pantalla reducida.

**-Validación de Usabilidad:** Se realizó una prueba de campo simulando la introducción de datos con una sola mano, confirmando que la disposición del botón flotante y el tamaño de los elementos del formulario cumplen con criterios de ergonomía y accesibilidad.

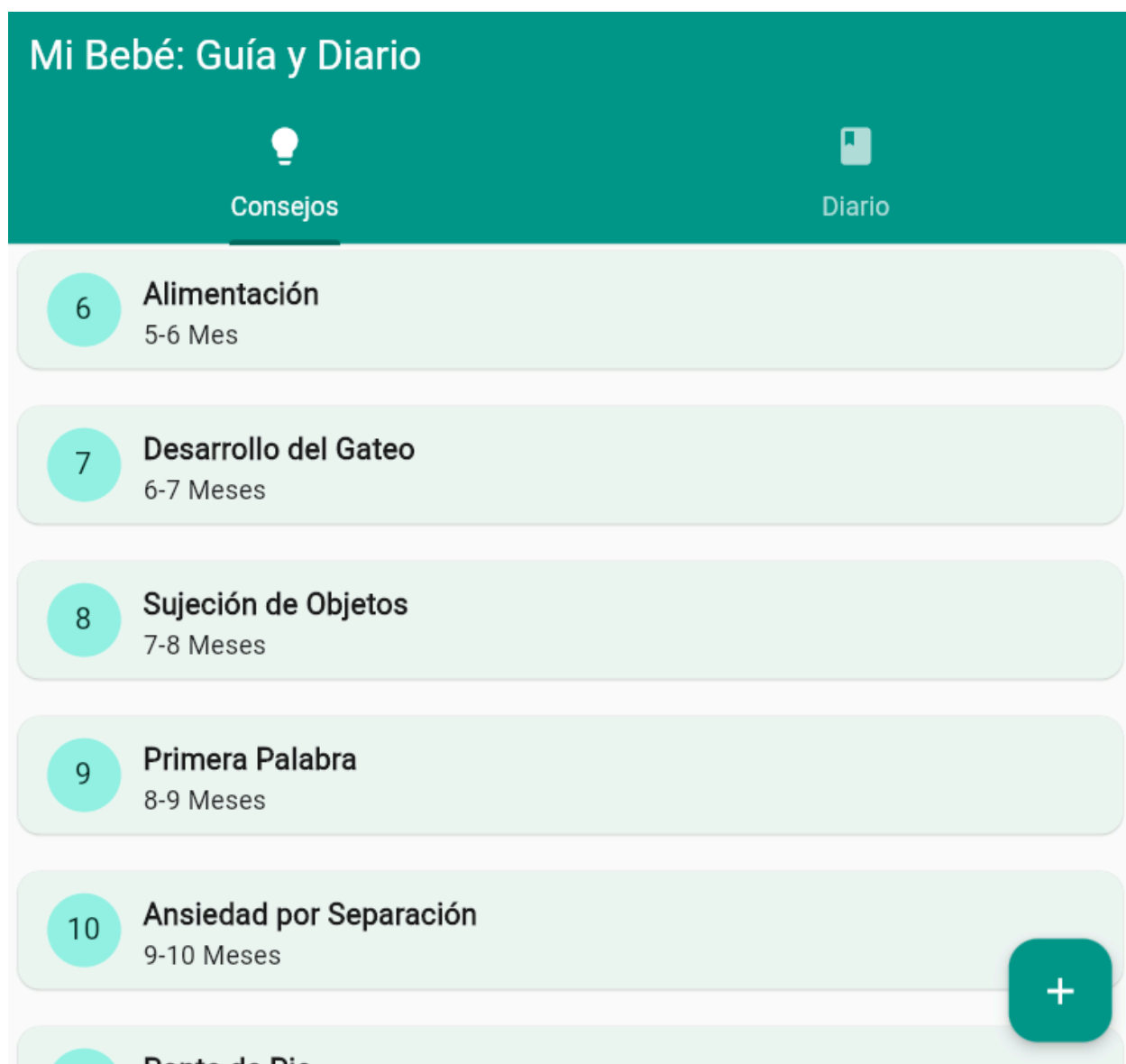


Figura 5: Navegación avanzada en la guía de desarrollo donde se aprecian los hitos correspondientes al segundo semestre de vida (6 a 10 meses), manteniendo la coherencia visual de la app.

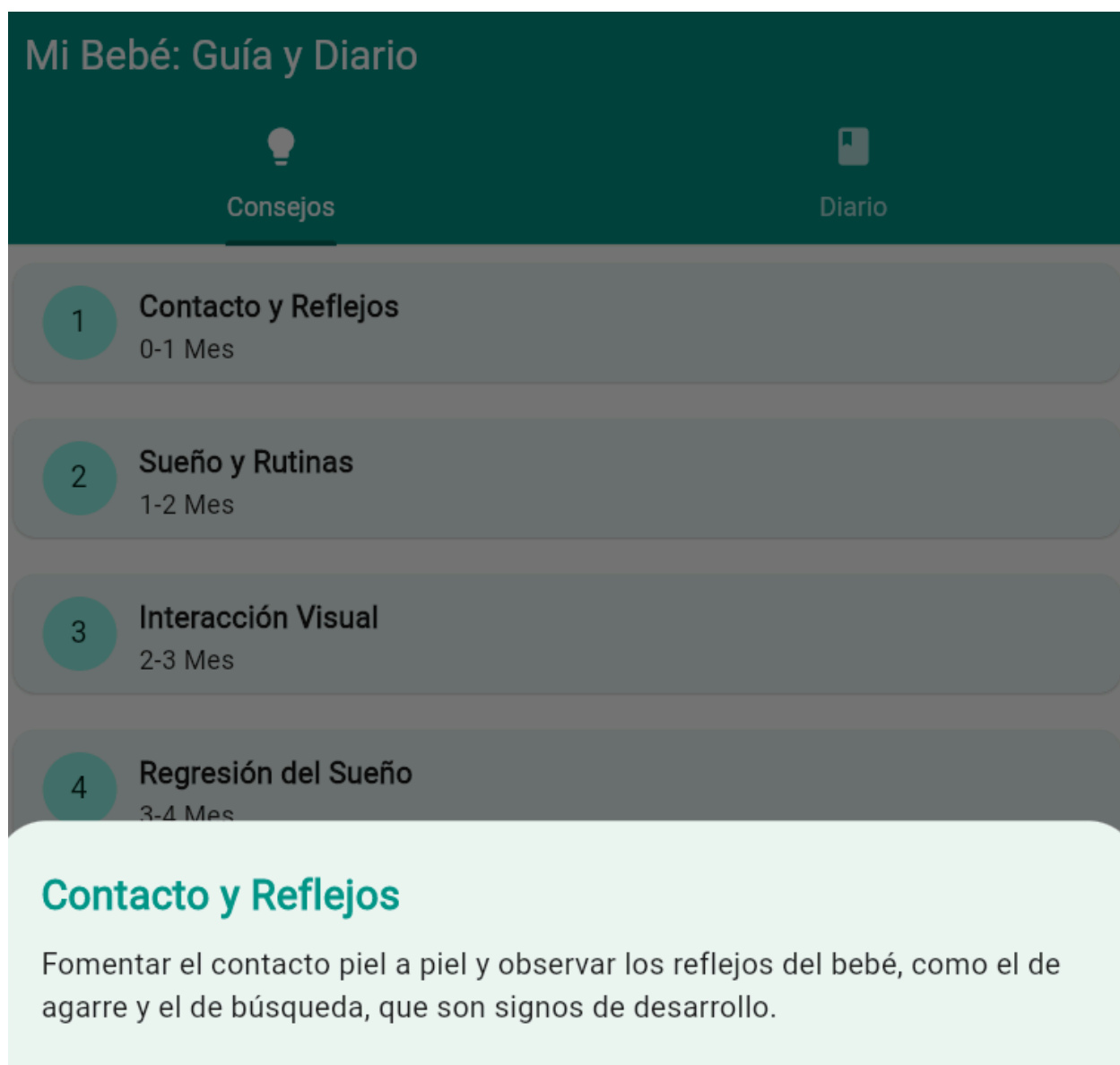


Figura 6: Detalle de la ventana modal descriptiva que se despliega al interactuar con un hito, ofreciendo información pedagógica extendida sin abandonar la pantalla principal.

## 6. CONCLUSIONES

Tras la finalización de este proyecto, la conclusión principal es que el desarrollo multiplataforma con **Flutter** representa una ventaja competitiva real para cualquier desarrollador actual. Lo que comenzó como una idea para ayudar en la gestión de la crianza, se ha materializado en una aplicación funcional que resuelve dos problemas críticos: el acceso rápido a información de calidad y el registro de datos en tiempo real.

Desde un punto de vista técnico, la implementación de **BabySteps** me ha permitido profundizar en la gestión de estados y en la creación de interfaces dinámicas. El mayor aprendizaje no ha sido solo escribir el código, sino entender la importancia de la arquitectura: separar la lógica de los datos (las clases Consejo y Registro) de la interfaz visual ha sido fundamental para que la app sea escalable y fácil de mantener.

En cuanto a la experiencia de usuario (UX), he comprobado que en aplicaciones de salud infantil, "menos es más". La decisión de integrar un diario sencillo y una guía directa, sin menús infinitos, es lo que aporta el verdadero valor añadido. La validación del sistema mediante las pruebas realizadas demuestra que es posible crear herramientas potentes que sean, a la vez, extremadamente sencillas de utilizar por personas que pueden estar bajo situaciones de fatiga o estrés.

Como trabajo futuro, queda abierta la posibilidad de implementar una base de datos local (como SQLite o Hive) para que los datos del diario persistan incluso si se borra la caché del navegador, así como la integración de notificaciones push para recordar las tomas o vacunas. En definitiva, este proyecto ha cumplido con los objetivos marcados, demostrando que con las herramientas tecnológicas adecuadas y un enfoque centrado en el usuario, se pueden desarrollar soluciones móviles de alta calidad en tiempos de desarrollo muy ajustados.

## 7. BIBLIOGRAFÍA

Para el desarrollo de este proyecto se han consultado las siguientes fuentes y recursos, asegurando el uso de estándares actuales en el desarrollo móvil:

- **Google Flutter Documentation (2026).** *Widgets Catalog and Material Design 3 guides:* <https://docs.flutter.dev/>
- **Dart Language Tour (2026).** *Language basics and types:* <https://dart.dev/guides/language/language-tour>
- **Zapp.run Documentation.** *Online Flutter Sandbox environment:* <https://zapp.run/docs>
- **Material Design 3 (M3).** *Design tokens and accessibility principles:* <https://m3.material.io/>
- **Intl Package for Dart.** *Internationalization and Date formatting:* <https://pub.dev/packages/intl>

## 8. ANEXOS

- **Anexo I:** Código fuente completo: <https://github.com/JLLR85/TSDAM-BabySteps.git>

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';

void main() => runApp(MaterialApp(
  debugShowCheckedModeBanner: false,
  theme: ThemeData(colorSchemeSeed: Colors.teal, useMaterial3: true),
  home: PantallaPrincipal()));

class Consejo { final String titulo, descripcion, etapa; Consejo(this.titulo, this.descripcion, this.etapa);
}

class Registro { final String tipo, hora, observacion; Registro(this.tipo, this.hora, this.observacion); }

class PantallaPrincipal extends StatefulWidget {
  @override _PantallaPrincipalState createState() => _PantallaPrincipalState();
}

class _PantallaPrincipalState extends State<PantallaPrincipal> {
  // Lista de hitos (se omiten entradas repetitivas para optimizar espacio en anexo)
  final List<Consejo> misConsejos = [
    Consejo("Contacto y Reflejos", "Fomentar el contacto piel a piel...", "0-1 Mes"),
    Consejo("Sueño y Rutinas", "Establecer una rutina consistente...", "1-2 Mes"),
    Consejo("Desarrollo Motor", "Iniciar el tiempo boca abajo...", "4-5 Mes"),
    // ... Hitos adicionales del 6 al 18 implementados según el modelo de datos
  ];

  final List<Registro> misRegistros = [];
  final TextEditingController _obsController = TextEditingController();
  String _tipoSel = 'Toma (Pecho/Bibe)';

  void _agregarRegistro() {
    setState(() {
      misRegistros.insert(0, Registro(_tipoSel, DateFormat('HH:mm').format(DateTime.now()),
_obsController.text));
      _obsController.clear();
    });
    Navigator.pop(context);
  }

  void _mostrarForm(BuildContext context) {
    showModalBottomSheet(context: context, isScrollControlled: true, builder: (c) => Padding(
      padding: EdgeInsets.only(bottom: MediaQuery.of(c).viewInsets.bottom, left: 20, right: 20, top: 20),
      child: Column(mainAxisSize: MainAxisSize.min, children: [
        const Text("Nuevo Registro", style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold)),
        DropdownButton<String>(value: _tipoSel, isExpanded: true,
```





```
items: ['Toma (Pecho/Bibe)', 'Sueño', 'Pañal',
```

```
'Medicación'].map((v) => DropdownMenuItem(value: v,
```

```
child: Text(v))).toList(),
```

```
onChanged: (n) => setState(() => _tipoSel = n!)),
```

```
TextField(controller: _obsController, decoration: const InputDecoration(labelText: "Notas")),
```

```
const SizedBox(height: 20),
```

```
ElevatedButton(onPressed: _agregarRegistro, child: const Text("GUARDAR")),
```

```
const SizedBox(height: 20),
```

```
]]));
```

```
}
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
return DefaultTabController(length: 2, child: Scaffold(
```

```
appBar: AppBar(title: const Text("BabySteps"), bottom: const TabBar(tabs: [Tab(text: "CONSEJOS"),
```

```
Tab(text: "DIARIO")])),
```

```
body: TabBarView(children: [
```

```
ListView.builder(itemCount: misConsejos.length, itemBuilder: (c, i) => Card(
```

```
child: ListTile(title: Text(misConsejos[i].titulo), subtitle: Text(misConsejos[i].etapa),
```

```
onTap: () => showModalBottomSheet(context: c, builder: (c) => Padding(
```

```
padding: const EdgeInsets.all(20), child: Text(misConsejos[i].descripcion)))))),
```

```
misRegistros.isEmpty ? const Center(child: Text("Sin registros")) :
```

```
ListView.builder(itemCount: misRegistros.length, itemBuilder: (c, i) => ListTile(
```

```
leading: const Icon(Icons.history), title: Text(misRegistros[i].tipo),
```

```
subtitle: Text(misRegistros[i].observacion), trailing: Text(misRegistros[i].hora)))
```

```
]),
```

```
floatingActionButton: FloatingActionButton(onPressed: () => _mostrarForm(context), child: const
```

```
Icon(Icons.add)),
```

```
));
```

```
}
```

```
}
```